

Фиксация основных идей и концепций, формируемых в ходе разработки программы

Задача об экзамене

Индивидуальное задание № 1. Вариант 17
«Архитектура компьютера и операционные системы»
НИУ ВШЭ, факультет компьютерных наук

Исполнитель : Бадретдинов Азат Рустэмович

Группа: БПИ252

Москва, 2026

Постановка задачи

Несколько преподавателей проводят единый письменный экзамен у группы студентов. Каждый студент последовательно получает уникальный билет, сообщает его номер, готовится в течение случайного времени, передаёт работу на проверку и ожидает результата. Свободный преподаватель проверяет не более одной работы одновременно, выставляет оценку и сообщает её соответствующему студенту.

Требуется консольная программа для Linux, моделирующая взаимодействие по схеме «клиент-сервер». По общим требованиям первого задания реализована последовательная имитация на C: студенты и преподаватели являются самостоятельными моделями внутри одной программы. Их взаимодействие выражается передачей работ через очередь и выдачей результатов.

Основные сущности программы

Экзамен (Exam)

Экзамен объединяет участников, очередь, параметры, генератор случайных чисел, условные часы и статистику. Он задаёт общий порядок событий и управляет завершением модели. Единица времени - такт. Подготовка и проверка занимают целое положительное число тактов.

Параметры (Config)

Объект хранит количество преподавателей, студентов и билетов, диапазон подготовки, длительность проверки, размер очереди, предел времени, правила выдачи билетов и оценки. Дополнительные настройки задают seed, задержку показа и путь журнала.

Участники и их поведение

Студент (Student)

Студент хранит состояние, момент окончания подготовки и свою работу. Индекс в массиве определяет уникальный номер студента. После выдачи билета в момент $t=0$ начинается подготовка. Готовый студент пытается передать работу в очередь. Если места нет, работа остаётся у него до освобождения места. После получения оценки участие заканчивается.

Состояние	Смысл
-----------	-------

NOT_ADMITTED	Не допущен: уникального билета не хватило.
PREPARING	Готовит ответ до момента ready_at.
READY	Закончил подготовку, работа ещё у студента.
QUEUED	Работа передана и находится в очереди.
CHECKING	Работу проверяет назначенный преподаватель.
DONE	Получил свою оценку и завершил экзамен.

Работа (Work) и билет

Работа содержит номер билета, оценку и номер проверяющего. Она вложена в структуру студента и не копируется при передаче. Очередь и преподаватель хранят индекс владельца, поэтому результат относится к нужному студенту. Нулевая оценка означает отсутствие результата.

Билеты представлены числами от 1 до заданного количества. Режим sequential выдаёт их по порядку; random предварительно перемешивает массив алгоритмом Фишера-Йетса. Билеты не повторяются. Если их меньше, чем студентов, допускаются первые $\min(\text{число студентов, число билетов})$ участников; остальные получают сообщение об отказе.

Преподаватель (Teacher)

Преподаватель хранит индекс текущего студента и время окончания проверки. Значение индекса -1 означает свободное состояние. При назначении работы окончание вычисляется как текущее время плюс длительность проверки. После выдачи оценки преподаватель освобождается.

Очередь (Queue)

Кольцевой буфер хранит индексы студентов, голову, размер и вместимость. Работы извлекаются по FIFO. Проверяемые работы не занимают мест в очереди. Среди готовых студентов, ещё не сдавших работу, первым выбирается тот, кто раньше закончил подготовку; при равенстве - студент с меньшим номером.

Правила развития модели

Начальные условия

После проверки аргументов открывается журнал и выделяется память. Всем допущенным студентам последовательно выдаются билеты. Выдача не расходует такты, поэтому подготовка каждого начинается в $t=0$. Её длительность выбирается из включительного диапазона $[\text{prep-min}, \text{prep-max}]$. Все преподаватели первоначально свободны.

Один такт экзамена

1. Проверить сигнал прерывания и ошибки записи.
2. Завершить проверки, срок которых наступил; выдать оценки владельцам работ, освободить преподавателей.
3. Отметить студентов, закончивших подготовку, как готовых к передаче работы.
4. Если результаты получены всеми допущенными студентами, завершить модель.
5. Если достигнут предел времени, завершить модель без новых назначений.
6. Назначить свободным преподавателям работы из существующей очереди.
7. Принять готовые работы при наличии места; после каждого принятия снова попытаться назначить работу свободному преподавателю.
8. Сообщить студентам, впервые столкнувшимся с полной очередью, об ожидании места. Выполнить необязательную задержку и увеличить время на один такт.

Формирование оценки

В режиме `random` оценка выбирается из 2, 3, 4, 5. В режиме `ticket` используется формула $2 + ((b - 1) \bmod 4)$, где b - номер билета. Это условное правило для демонстрации модели, а не оценивание реальных знаний. При одинаковых параметрах и `seed` журнал воспроизводим: используется собственный генератор `xorshift32`.

Завершение

Штатное окончание происходит после выдачи результатов всем допущенным студентам, включая случай нулевого допуска. При достижении максимального времени фиксируются состояния незавершённых работ. На самой границе времени уже закончившиеся проверки учитываются, новые не начинаются. Если все оценки получены на границе, причина завершения - `complete`.

Значение `max-time=0` снимает ограничение времени. `SIGINT` (Ctrl+C) и `SIGTERM` устанавливают флаг, после чего основной цикл печатает итог и освобождает память. В обработчике сигнала нет операций вывода или освобождения памяти.

Пользовательский интерфейс

Программа принимает аргументы командной строки. Без аргументов используется готовый демонстрационный набор. Команда `./exam --help` выводит справку. Все ключевые события имеют отметку условного времени и идентификаторы участников.

Аргумент	Назначение	По умолчанию
<code>--teachers</code>	Преподаватели: 1..1000	2
<code>--students</code>	Студенты: 0..10000	6
<code>--tickets</code>	Билеты: 0..10000	6
<code>--ticket-mode</code>	sequential / random	random
<code>--prep-min</code>	Минимум подготовки: 1..100000	2
<code>--prep-max</code>	Максимум подготовки: 1..100000	5
<code>--check-time</code>	Проверка: 1..100000 тактов	2
<code>--grade-mode</code>	random / ticket	random
<code>--queue</code>	Очередь: 1..10000 мест	3
<code>--max-time</code>	Предел: 0..100000; 0 - без предела	100
<code>--seed</code>	Seed: 1..2147483647	17
<code>--delay-ms</code>	Пауза такта: 0..60000 мс	0
<code>--log</code>	Путь файла журнала	exam.log

Нечисловые значения, выход за границы и обратный диапазон подготовки отклоняются до запуска модели. Число преподавателей и очередь должны быть положительными. Нулевое число студентов или билетов обрабатывается штатно.

Сборка и запуск в Linux / WSL

```
make
./exam
./exam --delay-ms 500
./exam --students 5 --tickets 2 --log shortage.log
./exam --max-time 1 --log timeout.log
./exam --max-time 0 --delay-ms 500
make test
./launch_scripts/showcase.sh
```

Для сборки нужны компилятор C и make. Для тестов нужен Python 3. Файл конфигурации включён в проект.

Файловый ввод-вывод и результаты

Журнал открывается через `open`, вывод реализован через `write`; обработаны короткая запись и `EINTR`. Вспомогательные сообщения используют `dprintf`. Терминал и лог получают одинаковые события. Файл по указанному пути перезаписывается при запуске. В конце выводятся состояния всех студентов, причина завершения, количество допущенных и завершивших, незавершённые работы, отказы, максимум очереди, число ожидавших места и средняя оценка.

Сохранение корректности

1. Уникальность билетов обеспечивается выдачей разных элементов массива. Число допущенных не превышает числа билетов.
2. Работа поступает в очередь только из READY. После помещения состояние меняется на QUEUED, поэтому повторная постановка невозможна.
3. Преподаватель получает работу только в свободном состоянии; при назначении студент переходит в CHECKING.
4. Преподаватель сохраняет индекс принятой работы до завершения. Оценка записывается в работу соответствующего студента.
5. После выдачи результата состояние DONE исключает дальнейшую подготовку, постановку в очередь и проверку этой работы.
6. При переполнении работа сохраняется у студента. При остановке перечисляются все текущие состояния, поэтому незавершённая работа остаётся учтённой.

Структура реализации

Файл	Ответственность
src/main.c	Запуск разбора параметров и моделирования.
src/exam.h	Структура Config и объявления функций.
src/config.c	Значения по умолчанию, справка, проверки аргументов.
src/exam.c	Участники, очередь, такты, сигналы, журнал и статистика.
tests/test_exam.py	Автоматические сценарии и проверка событий.
launch_scripts/showcase.sh	Демонстрация четырёх ситуаций и сохранение журналов.
results/	Фактические журналы и сводки тестирования.

Оценка затрат

Кольцевая очередь выполняет добавление и извлечение за $O(1)$. Каждый такт просматривает студентов и преподавателей. Выбор старейшей готовой работы линейный; суммарно по всем принятым работам он может потребовать $O(S^2)$. Общая верхняя оценка времени - $O(T(S+P)+S^2+B)$, памяти - $O(S+P+Q+B)$, где S - студенты, P - преподаватели, Q - очередь, B - билеты, T - число тактов. Для больших временных диапазонов эффективнее событийная модель с переходом сразу к ближайшему событию; здесь выбран более простой наглядный цикл.

Результаты работы и тестирование

Программа собрана с C11 и флагами -Wall -Wextra -Wpedantic -Werror. Выполнены 30 автоматических проверок. Все завершились успешно. Журналы каждого сценария сохранены в каталоге results.

Сценарий	Фактический результат
По умолчанию	6 из 6 завершили в t=8; средняя оценка 3,67; максимум очереди 3; один студент ждал места.
Повтор с seed=17	Журнал полностью совпал с первым запуском.
1 преподаватель, 3 студента	Очередь на 1 место: все завершили в t=7; оценки 2, 3, 4; один студент ждал места.
5 студентов, 2 билета	Допущены и завершили 2; отказано 3.
0 студентов / 0 билетов	Штатное завершение без зависания; во втором случае 6 отказов.
Предел 1 такт	0 завершили, 6 незавершённых; причина timeout.
Окончание на границе	1 студент получил результат в t=3 при пределе 3; причина complete.
Без предела времени	Все допущенные студенты получили результат.
9 ошибочных наборов	Отклонены с кодом 2.
10 нагрузочных наборов	По 40 студентов, очередь на 1 место, разные seed и число преподавателей; все завершили.
SIGINT / SIGTERM	Итог сохранён; коды выхода 130 и 143.

Тесты проверяют отсутствие повторных билетов и проверок, невозможность двух одновременных работ у преподавателя, соответствие принятой и законченной работы, диапазон оценок, вместимость очереди и баланс итоговой статистики. Вывод успешных запусков сравнивается с файлом журнала.

Разбор наглядного запуска

Пусть есть один преподаватель, три студента и три билета. Подготовка каждого занимает один такт, проверка - два такта, очередь вмещает одну работу. Выдача билетов последовательная, оценка определяется номером билета.

```
./exam --teachers 1 --students 3 --tickets 3 \  
--ticket-mode sequential --grade-mode ticket \  
--prep-min 1 --prep-max 1 --check-time 2 --queue 1
```

Время События фактического запуска

0	Студенты получают билеты 1, 2, 3 и начинают подготовку.
1	Все готовы. Работа 1 сразу назначается преподавателю. Работа 2 остаётся в очереди. Студент 3 ждёт места, сохраняя работу у себя.
3	Студент 1 получает оценку 2. Преподаватель берёт работу 2. Студент 3 передаёт работу в освободившуюся очередь.
5	Студент 2 получает оценку 3. Начинается проверка работы 3.
7	Студент 3 получает оценку 4. Все допущенные завершили экзамен.

Итог: admitted=3, completed=3, unfinished=0, rejected=0, peak_queue=1, waited=1, average=3.00. Полный журнал - results/fixed.log.

Использование ИИ

ChatGPT использовался для подготовки автоматических тестов на Python и файла README с описанием запросов.

Запросы к ИИ

1. «Напиши тесты для моего проекта». Результат: автоматические тесты в tests/test_exam.py.
2. «Напиши мне readme файл с промтами». Результат: файл README_AI.md.

Критическая оценка результатов

Подготовленные тесты проверяют уникальность билетов, отсутствие повторной проверки работ, вместимость очереди, соответствие результата владельцу работы и итоговую статистику. Также проверяются некорректные параметры, недостаточное число билетов, завершение по времени и обработка сигналов.

Тесты анализируют текстовый вывод программы, поэтому изменение формата сообщений может потребовать их корректировки. Проверки охватывают выбранные сценарии.